IT093IU – Web Application Development

Instructor: MSc. Nguyen Trung Nghia

INVENTORY MANANGEMENT SYSTEM

Final Project Presentation

Team Members:

1. Lê Thái Minh Tín (ITCSIU24086)
2. Huỳnh Ngọc Thủy Tiên (ITITIU24065)

TABLE OF CONTENTS

01 Introduction

02 Demonstration

03 Technical Deep-Dive

04 Results & Reflection

THE PROBLEM & MARKET GAP

The Problem:

- Inventory shrinkage
- Operational bottlenecks
- Financial blind spots

The Cause:

- Fragmented systems lead to lost track of stock
- Missing automated alerts cause stockouts and lost revenue

THE PROBLEM & MARKET GAP

Facets	BlueYonder	OdooWMS	Shopee Management Channel
Strength	Enterprise-scale; End-to-end integration; Financial Ledge	Open-source DB; Modularity	Simple; User-friendly UI
Weakness	Costly to deploy; Out-of-scope	High requirements for modification	Limited depth

SOLUTION

- Dashboard Analytics
- Durable Master Data
- Automated Workflows
- Proposals & In-App Notifications
- Role-Based Security & Auditing

TECHNOLOGY STACK

Frontend	Backend	Database
React 19.2.0	Spring Boot 3.3.5	MySQL 8+
Tailwind CSS	Java 21	Raw JDBC
	JWT Auth	

INFRASTRUCTURE & DEPLOYMENT

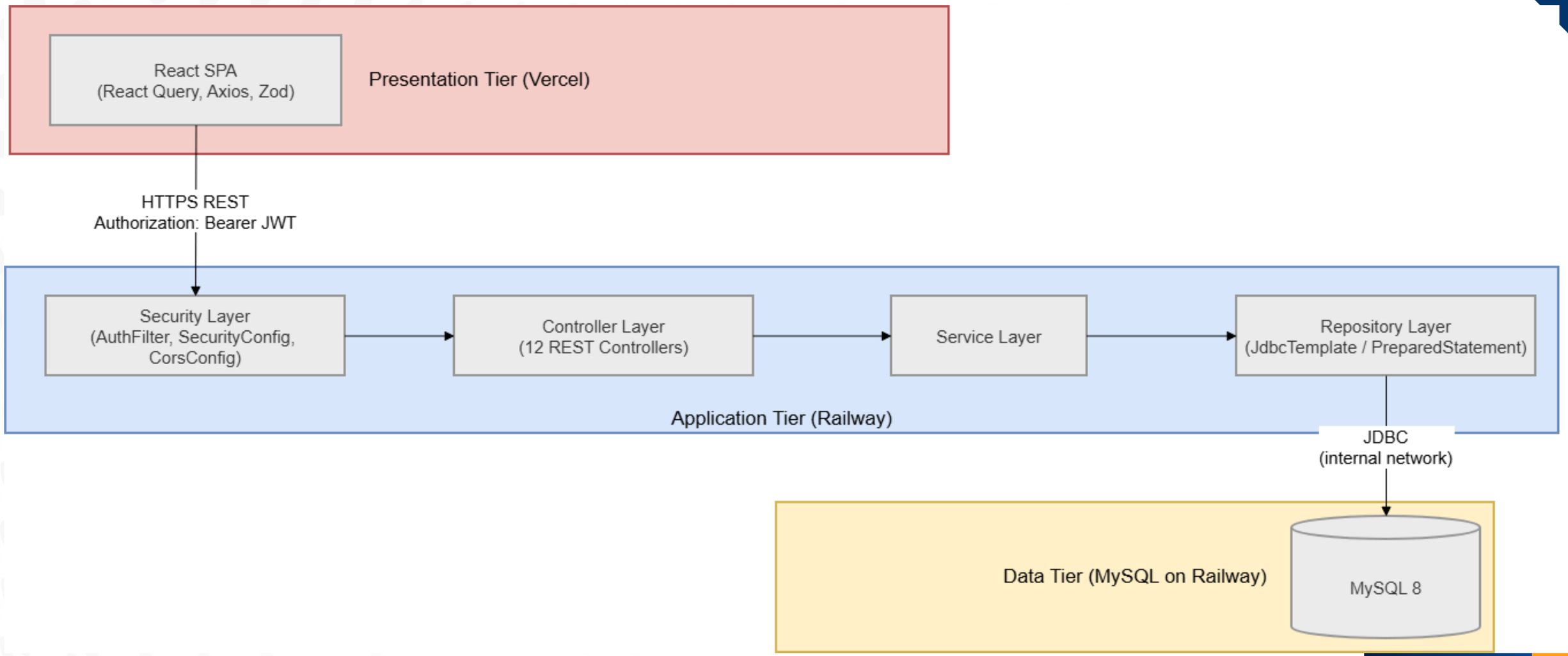
Hosted on Vercel

Hosted on Railway

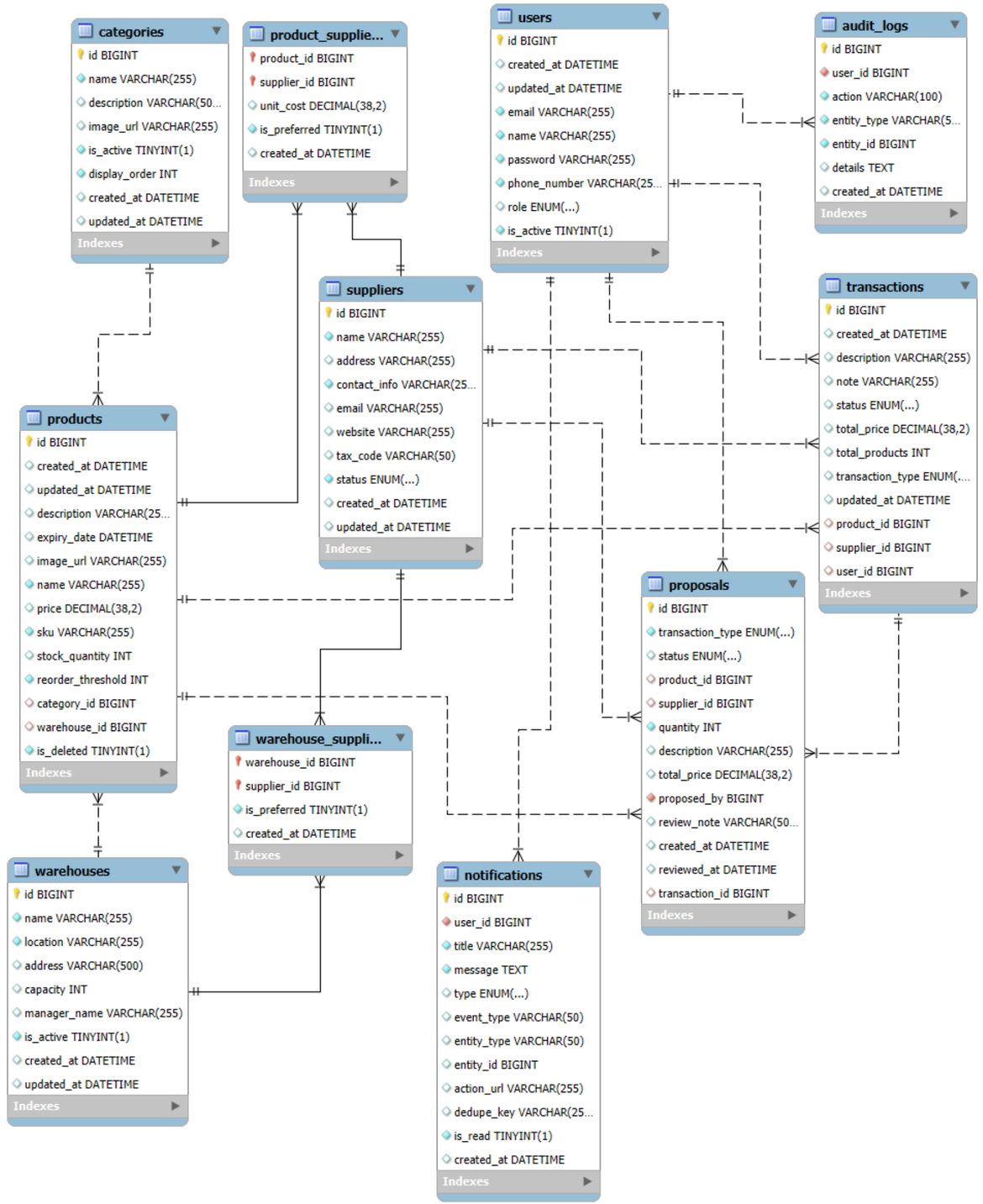
02.
DEMO

3. TECHNICAL DEEP-DIVE

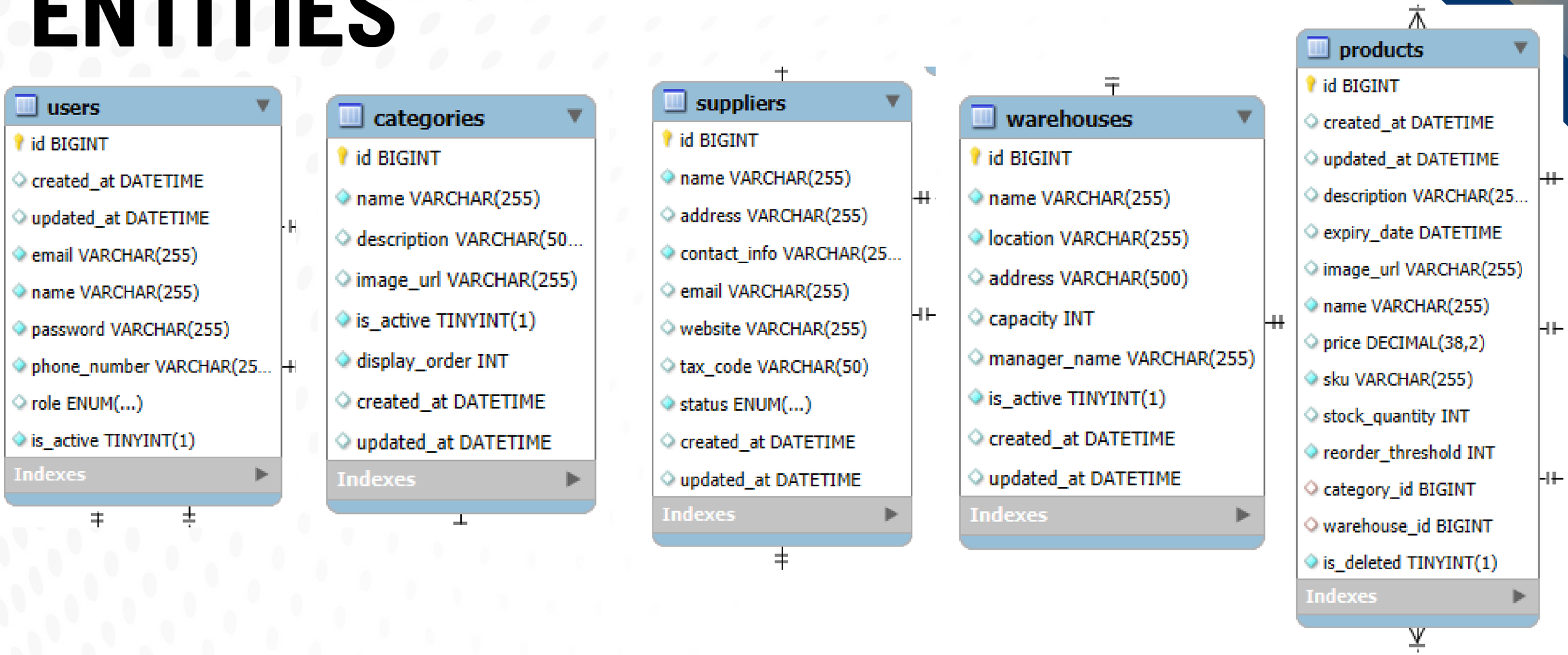
SYSTEM ARCHITECTURE DIAGRAM



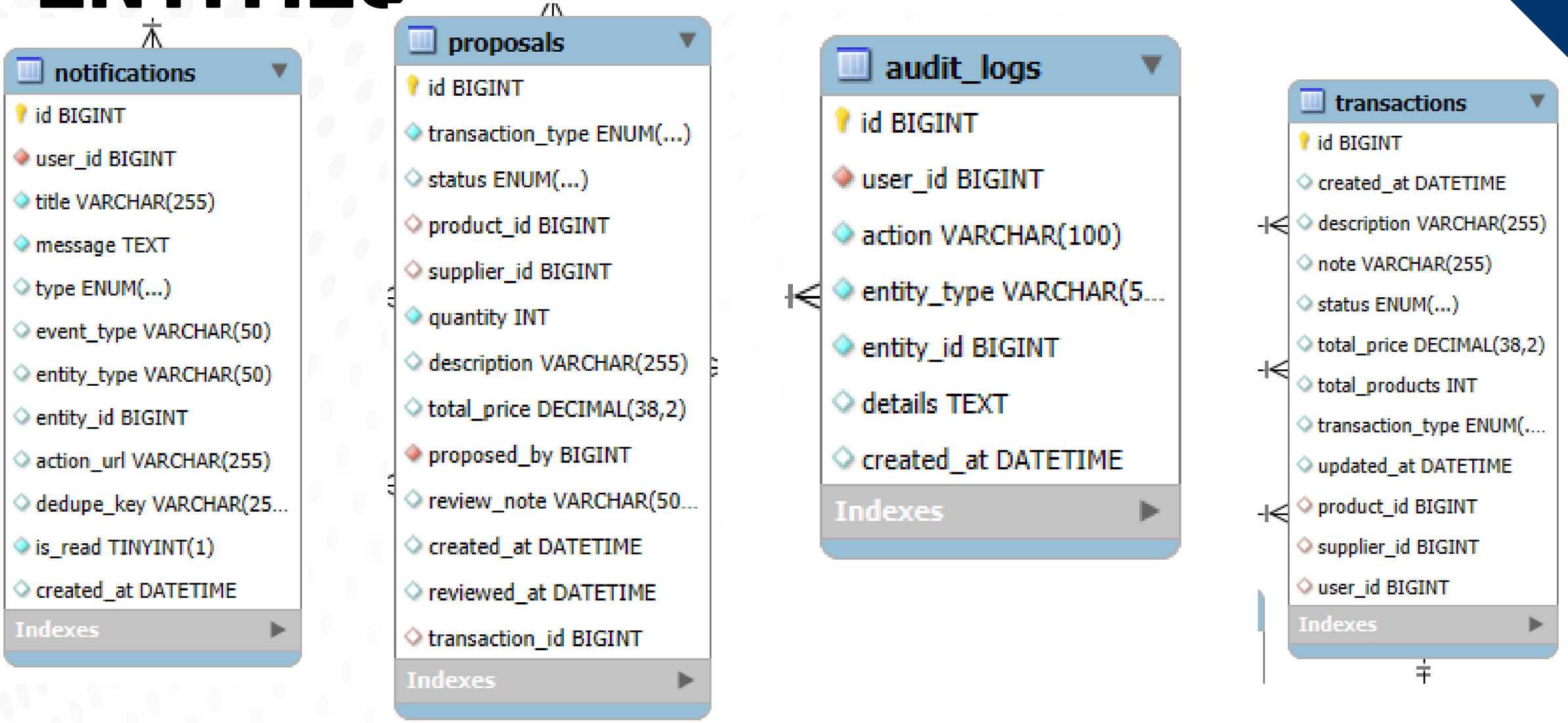
DATABASE SCHEMA



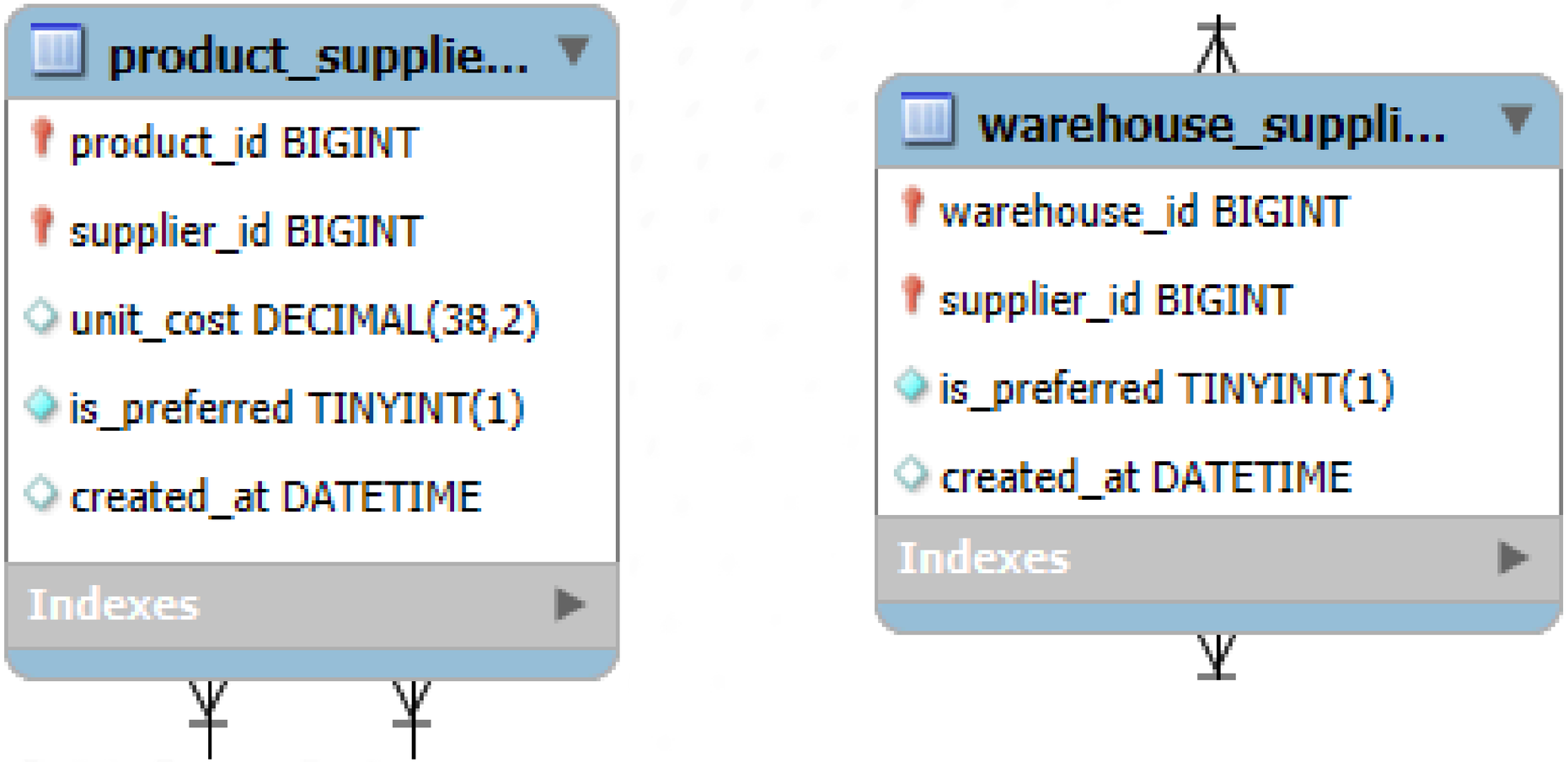
ENTITIES


users
id BIGINT
created_at DATETIME
updated_at DATETIME
email VARCHAR(255)
name VARCHAR(255)
password VARCHAR(255)
phone_number VARCHAR(25...
role ENUM(...)
is_active TINYINT(1)
Indexes
categories
id BIGINT
name VARCHAR(255)
description VARCHAR(50...
image_url VARCHAR(255)
is_active TINYINT(1)
display_order INT
created_at DATETIME
updated_at DATETIME
Indexes
suppliers
id BIGINT
name VARCHAR(255)
address VARCHAR(255)
contact_info VARCHAR(25...
email VARCHAR(255)
website VARCHAR(255)
tax_code VARCHAR(50)
status ENUM(...)
created_at DATETIME
updated_at DATETIME
Indexes
warehouses
id BIGINT
name VARCHAR(255)
location VARCHAR(255)
address VARCHAR(500)
capacity INT
manager_name VARCHAR(255)
is_active TINYINT(1)
created_at DATETIME
updated_at DATETIME
Indexes
products
id BIGINT
created_at DATETIME
updated_at DATETIME
description VARCHAR(25...
expiry_date DATETIME
image_url VARCHAR(255)
name VARCHAR(255)
price DECIMAL(38,2)
sku VARCHAR(255)
stock_quantity INT
reorder_threshold INT
category_id BIGINT
warehouse_id BIGINT
is_deleted TINYINT(1)
Indexes

ENTITIES


notifications
id BIGINT
user_id BIGINT
title VARCHAR(255)
message TEXT
type ENUM(...)
event_type VARCHAR(50)
entity_type VARCHAR(50)
entity_id BIGINT
action_url VARCHAR(255)
dedupe_key VARCHAR(25...
is_read TINYINT(1)
created_at DATETIME
Indexes
proposals
id BIGINT
transaction_type ENUM(...)
status ENUM(...)
product_id BIGINT
supplier_id BIGINT
quantity INT
description VARCHAR(255)
total_price DECIMAL(38,2)
proposed_by BIGINT
review_note VARCHAR(50...
created_at DATETIME
reviewed_at DATETIME
transaction_id BIGINT
Indexes
audit_logs
id BIGINT
user_id BIGINT
action VARCHAR(100)
entity_type VARCHAR(5...
entity_id BIGINT
details TEXT
created_at DATETIME
Indexes
transactions
id BIGINT
created_at DATETIME
description VARCHAR(255)
note VARCHAR(255)
status ENUM(...)
total_price DECIMAL(38,2)
total_products INT
transaction_type ENUM(...
updated_at DATETIME
product_id BIGINT
supplier_id BIGINT
user_id BIGINT
Indexes

JUNCTION TABLE



RELATIONSHIP

```
CREATE TABLE products (
    ...
    FOREIGN KEY (category_id)  REFERENCES categories(id)  ON DELETE CASCADE  ON UPDATE CASCADE,
    FOREIGN KEY (warehouse_id) REFERENCES warehouses(id)  ON DELETE SET NULL ON UPDATE CASCADE,
    FOREIGN KEY (created_by)   REFERENCES users(id)       ON DELETE SET NULL ON UPDATE CASCADE,
    FOREIGN KEY (updated_by)   REFERENCES users(id)       ON DELETE SET NULL ON UPDATE CASCADE,
    CHECK (stock_quantity >= 0)
);
```

ON DELETE CASCADE: Category is dropped → orphaned products are removed.

ON DELETE SET NULL: warehouse is removed → product records are preserved

STRATEGY

INDEXING

- Index the is_deleted and status columns.
- Index all foreign key columns

SOFT-DELETION

- Mark as inactive, NOT permanently deleted
- Ensure historical transaction logs

CODE HIGHLIGHT

CLIENT-SIDE VALIDATION

```ts
export const userSchema = z.object({
  name: z.string().min(1, 'Name is required'),
  email: z
    .string()
    .trim()
    .optional()
    .refine(
      (v) => !v || /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(v),
      'Invalid email format'
    ),
  phoneNumber: z.string().min(1, 'Phone number is required'),
  password: z.string().trim().optional(),
  role: z.enum(['ADMIN', 'MANAGER', 'STAFF']).default('STAFF'),
});
```

React Hook Form

```ts
const {
  register,
  handleSubmit,
  reset,
  formState: { errors, isSubmitting },
} = useForm({
  resolver: zodResolver(userSchema),
  defaultValues,
});
```

Zod schemas

CLIENT-SIDE VALIDATION

Catch user errors

Improve performance and user experience

```
export const userSchema = z.object({
  name: z.string().min(1, 'Name is required'),
  email: z
    .string()
    .trim()
    .optional()
    .refine(
      (v) => !v || /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(v),
      'Invalid email format'
    ),
  phoneNumber: z.string().min(1, 'Phone number is required'),
  password: z.string().trim().optional(),
  role: z.enum(['ADMIN', 'MANAGER', 'STAFF']).default('STAFF'),
});
```

```
const {
  register,
  handleSubmit,
  reset,
  formState: { errors, isSubmitting },
} = useForm({
  resolver: zodResolver(userSchema),
  defaultValues,
});
```

ROLE-AWARE SERVICE DISPATCH

STAFF

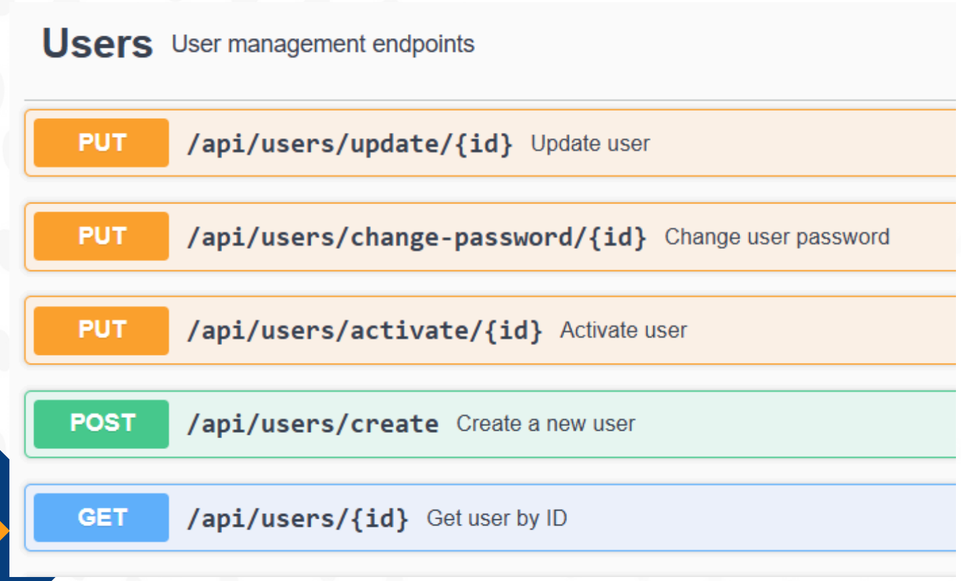
```
public class TransactionServiceImpl implements TransactionService {
        public Response restockInventory(TransactionRequest transactionRequest) {

            User user = userService.getCurrentLoggedInUser();

            // STAFF must create a proposal instead of executing directly
            if (user.getRole() == UserRole.STAFF) {
                transactionRequest.setTransactionType(TransactionType.PURCHASE);
                return proposalService.createProposal(transactionRequest);
            }
```

MANAGER

```
int currentStock = getCurrentStock(product);
Integer previousStockQuantity = product.getStockQuantity();
product.setStockQuantity(currentStock + quantity);
productRepository.update(product);
```

CHALLENGES OVERCOME

Frontend-Backend Synchronization	check the Swagger to verify



Version Control Conflicts	use VS Code's visual merge tools

AI INTEGRATION & ENHANCEMENT

Tool

Claude and Gemini Code Review.

Comparison Results – Inventory Management System vs PROJECT_REQUIREMENTS.md

Audit Date: 2026-05-31 | Code changes since last audit: none (only docs/PLAN.md modified)

6.1 Functionality & Features – ~28/30

Core Features (18 pts)

User Authentication & Authorisation – ~3.75/4
- Registration: ✅ /api/auth/register – AuthController.java
- Login/logout (JWT): ✅ JwtUtils.java, AuthFilter.java, stateless session
- 3+ distinct roles: ✅ ADMIN, MANAGER, STAFF – UserRole.java, SecurityConfig.java:64
- RBAC enforced: ✅ @PreAuthorize on all mutating endpoints, SecurityConfig.java:47–95
- Profile management / change password: ✅ PUT /api/users/change-password/{id}, ProfilePage.jsx
- Admin user management dashboard: ✅ UserManagementPage.jsx – list, role change, activate/deactivate, delete
- Email verification: ⚠️ Not implemented – requirement says "recommended", not mandatory

CRUD Operations Completeness – 5/5
- ✅ Full CRUD on: Products, Categories, Suppliers, Warehouses, Transactions (create+status update), Users (update role+activate+delete), Notifications, Proposals, Audit Logs
- ✅ Entities ≥ 6–8 attributes: Products=15 cols, Suppliers=10 cols, Warehouses=9 cols, Transactions=12 cols
- ✅ Client-side: React Hook Form + Zod schemas per entity
- ✅ Server-side: @Size, @Email, @NotNull on RegisterRequest, business rules in services
- ✅ Cascading: categories→products ON DELETE CASCADE, products→product_suppliers ON DELETE CASCADE
- ✅ Soft delete: products.is_deleted, users.is_active
- ✅ Audit trail: created_at/updated_at/created_by/updated_by on all mutable tables
- ✅ Confirmation dialogs before all destructive actions (ConfirmationModal.jsx)

Search, Filter, Pagination – ~3.5/4

Tool

Claude and Gemini Code Review.

gemini-code-assist Bot reviewed last week

View reviewed changes

gemini-code-assist Bot left a comment

Code Review

This pull request introduces code splitting in the React application by lazy-loading all page components and wrapping the routes in a `Suspense` boundary with a fallback `PageLoader`. While this improves initial bundle size, placing a single `Suspense` boundary at the root level causes the entire layout (including navigation and sidebars) to unmount and flicker during route transitions. It is recommended to place nested `Suspense` boundaries around the `Outlet` component inside the layout wrapper to preserve the persistent UI components.

Developer Correction: Fix thread-safety and concurrency issues.

Hey, I'm currently a beginner in developing web applications and I want to implement a notification system, but I'm not sure what to adopt since I have 0 experience. Could you help me suggest something first for me to follow along?

Great question! For a beginner, I'd recommend using **Server-Sent Events (SSE)** — it's simpler than WebSockets and perfect for one-way real-time notifications (server pushing to client).

Here's why SSE is a good starting point:

- Built into HTTP, no special protocol needed
- Spring Boot has native support via `SseEmitter`
- The browser handles reconnection automatically
- Much less setup than WebSockets

Developer Correction: Fix thread-safety and concurrency issues.

Hm, I prefer a detailed implementation guide so I can follow along.

Absolutely! Here's a step-by-step implementation guide for a basic SSE notification system in Spring Boot.

Step 1 — Add Dependencies

Make sure your `pom.xml` has the Spring Web dependency:

xml

```xml
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

Developer Correction: Fix thread-safety and concurrency issues.

Step 2 — Create the Notification Controller

This is the core of your SSE implementation:

java

```java
@RestController
@RequestMapping("/api/notifications")
public class NotificationController {

    // Keeps track of all connected clients
    private final List<SseEmitter> emitters = new ArrayList<>();
```

Developer Correction: Fix thread-safety and concurrency issues.

```
public class NotificationStreamServiceImpl implements NotificationStreamService {

    private static final long TIMEOUT_MS = 30L * 60L * 1000L;
    private static final long HEARTBEAT_INTERVAL_MS = 20_000;
    private static final String KEEP_ALIVE_COMMENT = "keep-alive";

    private final Map<Long, CopyOnWriteArrayList<SseEmitter>> emittersByUserId = new ConcurrentHashMap<>();
```

04.
RESULTS AND REFLECTION

Key Contents

1. Accomplishments
2. Metrics
3. Reflection
4. Room for Improvement

ACCOMPLISHMENTS

Complete Web Application

Designed a system with appropriate technologies, clean architecture and complex business logic.

Complex Database Tables

Utilizing hand-written JDBC for control, proper optimization and business logic at the data layer.

Scalable & Secure Online System

Deployed to both VPS and Vercel/Railway with proper CI/CD pipeline.

Integration of Advanced Features

Integrated a professional proposal pipeline with user management.

METRICS

Database Table

11 tables (2 junctions) with proper indexing

Features

Basic CRUD features with 3 advanced domain feature and user management

Lines of Code & Commits

Approximately 70,000 lines of code across 253 commits.

LESSON LEARNED

Raw JDBC forces genuine SQL fluency.

Every query should have a justifiable design.

A justified architecture choice matters.

Usage of stateless JWT eliminates the root reason behind CSRF.

Deployment environment matters as much as code quality.

Migrating to a stable deployment is justified for reliability.

Active awareness on trade-offs

Acknowledging limitations and constant documentation is a genuine engineering discipline.

FUTURE WORK

Automated Reordering

Reordering workflows is currently manual, calling for an automatic purchasing pipeline.

WebSocket instead of Server-sent Events for Notifications

Enables instant communication among related parties.

Caching and Query Optimisation for the Dashboard

Introducing Redis would substantially save resources for database.

Offline-Capable Mobile Interface

Progressive Web App (PWA) or dedicated mobile client with a local-first data store is recommended for a non-disruptive experience.

Q&A SECTION

THANK YOU

FOR YOUR ATTENTION!